DSAI4203 Machine Learning
Mid-term Test

Part A: Multiple Choice Questions [Total: 30 marks]
Choose the **BEST** answer (1 only) for each question and write down your choice in the answer box below.

Answer box of Part A.

Question	1	2	3	4	5	6
Your Answer	B	C	B	A	B	B

1. In the context of decision trees, what does information gain measure?
   A. The increase in the number of branches after a split.
   B. The reduction in entropy after a dataset is split on an attribute.
   C. The total number of leaves in the tree.
   D. The average depth of the tree.

2. Which of the following statements is true about the k-nearest neighbours (kNN) algorithm?
   A. kNN requires a training phase to build a predictive model.
   B. kNN can only be used for regression tasks.
   C. The value of k determines how many neighbours are considered when making a prediction.
   D. kNN is not affected by the scale of the features in the dataset.

3. Suppose a simple MLP has an output $y = f(Wx + b)$, where W is the weight matrix, x is the input vector, b is the bias, and f is the activation function. If the loss function is $L(y,t)$, where t is the target, which expression represents the gradient of the loss with respect to the weights W?
   A. $\frac{\partial L}{\partial W} = x$
   B. $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot f'(Wx + b) \cdot x^T$
   C. $\frac{\partial L}{\partial W} = f(Wx + b) \cdot t$
   D. $\frac{\partial L}{\partial W} = b \cdot x$

4. Which of the following statements best describes the universal approximation theorem in relation to Multi-Layer Perceptrons (MLPs)?
   A. An MLP with a single hidden layer can approximate any continuous function, given enough neurons.
   B. An MLP can only approximate linear functions, regardless of its size.
   C. The universal approximation theorem states that MLPs require infinite layers to approximate any function.
   D. MLPs cannot approximate functions with more than two variables.

5. Which of the following best describes the role of convolutional filters (kernels) in a CNN?
   A. They randomly shuffle the input pixels to increase data diversity.
   B. They learn to detect local patterns by sliding over the input image and computing dot products.
   C. They compress the input image to reduce its size before classification.
   D. They convert the input image into a one-dimensional vector for fully connected layers.

6. Which of the following best describes the purpose of pooling layers (such as max pooling) in a CNN?

A. To increase the spatial resolution of feature maps.
B. To reduce the spatial dimensions of feature maps and make the representations more robust to small translations.
C. To apply non-linear activation functions to the feature maps.
D. To learn new features by updating weights during training,

Part B: Short and Long Questions [Total: 70 marks]
Fit your answer to the answer box provided for each question.

B1. [16 marks] Consider the figure below showing the decision boundary of *k*NN for $k = 1$ (left) and $k = 15$ (right) on a two-class problem and answer the following questions.

(a) Explain how the *k*NN algorithm determines the class of a new input point.
(b) Why do we say that the *k*NN may overfit for very small values of *k*, while it may underfit for very large values of *k*? Discuss this and describe in general how the value of *k* affects the shape and smoothness of the decision boundary created by *k*NN.
(c) If *k* is equal to the number of points in the training set, what is a *k*NN classifier equivalent to?

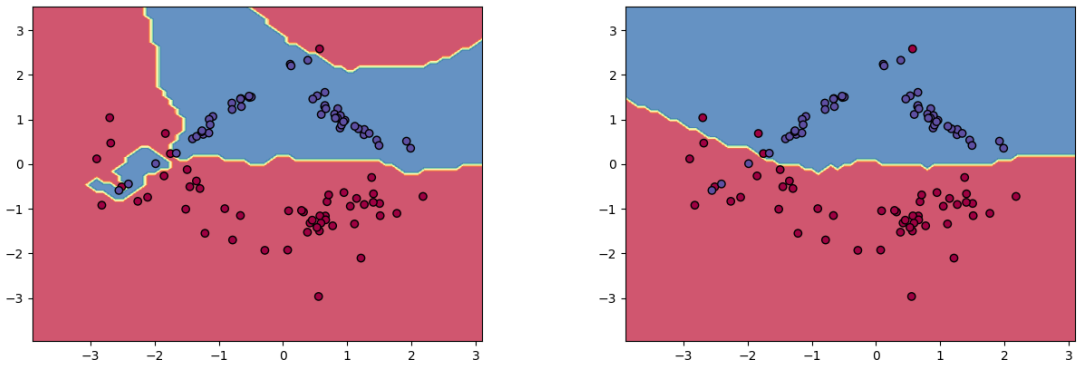


Figure 1: *k*NN with $k = 1$ (left) and with $k = 15$ (right).

Answer box of B1

(a) The kNN algorithm determines the class of a new input point by following these steps:
- calculate distances: It computes the distance (usually Euclidean) between the new point and all points in the training dataset.
- find nearest neighbours: It identifies the k training points that are closest to the new point.
- majority vote: It looks at the classes of these k nearest neighbours and assigns the new point to the class that appears most frequently among them. **[6 marks]**

(b) kNN may overfit for very small values of k because the algorithm bases its prediction for each new point solely on its nearest neighbour. This makes the model highly sensitive to noise and outliers in the training data, resulting in a very complex and irregular decision boundary that closely follows the training data. This may lead to mistakes in classifying new data.
On the other hand, kNN may underfit for very large values of k because it considers a large number of neighbours for each prediction. The decision is dominated by the overall majority class in the neighbourhood, which can smooth out important local patterns and ignore small clusters or subtle distinctions between classes. This leads to a very simple and smooth decision boundary that may fail to capture the true structure of the data. **[8 marks]**

(c) A majority vote classifier. **[2 marks]**

B2. [18 marks] You are building a decision tree classifier to determine whether a fruit is *Sweet* or *Not Sweet* based on two features: *Color* (Red or Green) and *Size* (Big or Small). You have the following dataset:

Fruit	**Color**	**Size**	**Sweet?**
1	Red	Big	Yes
2	Red	Small	No
3	Green	Big	No
4	Green	Small	No
5	Red	Big	Yes
6	Green	Big	Yes

Calculate the information gain for splitting on *Color* and for splitting on *Size*. Which feature should the decision tree split on first? Show your calculations.

Answer box of B2

There are 6 fruits, 3 sweet and 3 not sweet, so $p_{yes} = \frac{3}{6} = 0.5, \quad p_{no} = 0.5$

The initial entropy is $-p_{yes}\log_2(p_{yes}) - p_{no}\log_2(p_{no}) = -0.5\log_2(0.5) - 0.5\log_2(0.5) = 1$

If we split on Color, the entropy for Red (2 sweet, 1 not) is $-\frac{2}{3}\log_2\left(\frac{2}{3}\right) - \frac{1}{3}\log_2\left(\frac{1}{3}\right) \approx 0.918$ while the entropy for Green (1 sweet, 2 not) is $-\frac{1}{3}\log_2\left(\frac{1}{3}\right) - \frac{2}{3}\log_2\left(\frac{2}{3}\right) \approx 0.918$.

So the expected value of the entropy after splitting based on Color is $\frac{3}{6} \times 0.918 + \frac{3}{6} \times 0.918 = 0.918$ which gives the information gain $\text{IG(Color)} = 1 - 0.918 = 0.082$

[**8 marks**]

If we split on Size, the entropy for Big (3 sweet, 1 not) is $-\frac{3}{4}\log_2\left(\frac{3}{4}\right) - \frac{1}{4}\log_2\left(\frac{1}{4}\right) \approx 0.811$ while the entropy for Small (0 sweet, 2 not) is $-0\log_2(0) - 1\log_2(1) = 0$.

The expected entropy is $\frac{4}{6} \times 0.811 + \frac{2}{6} \times 0 = 0.541$ and thus $\text{IG(Size)} = 1 - 0.541 = 0.459$.

[**8 marks**]

Based on this information, the decision tree should split on Size first, because it gives the highest information gain.

[**2 marks**]

B3. [18 marks] Assume that you are given the two-class classification problem shown in Figure 2. Draw the decision boundary or boundaries for the following neural networks, giving a brief explanation of your answer.

(a) A neural network with 2 inputs and one output neuron.
(b) A neural network with 2 inputs, one hidden layer, and one output neuron, where the hidden layer has 2 nodes with sigmoid non-linearity.
(c) A neural network with 2 inputs, two hidden layers, and one output neuron, where each hidden layer has 2 nodes with sigmoid non-linearity.

You can assume that all the output neurons have sigmoid activation functions.

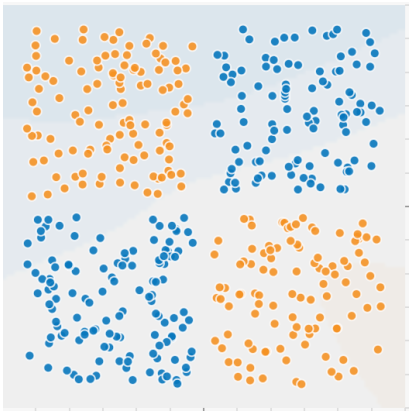
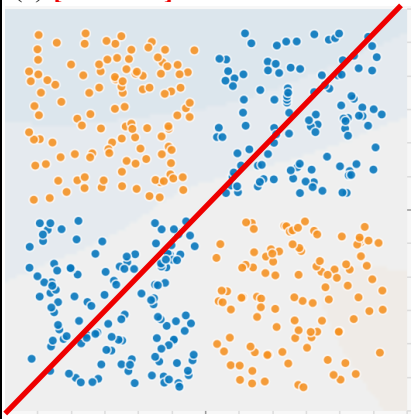


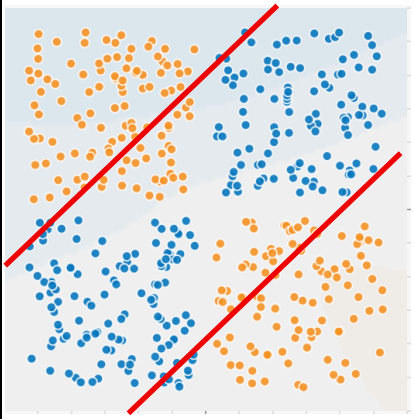
Figure 2: two-class classification problem.

Answer box of B3

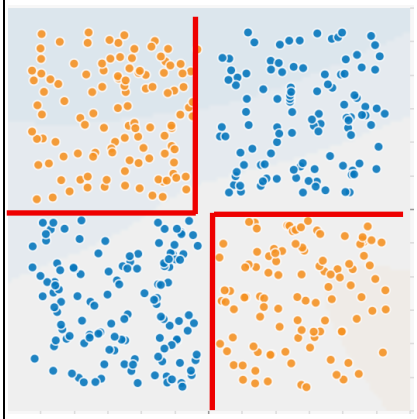
(a) **[3 marks]**



(b) **[3 marks]**



(c) **[3 marks]**



Explanations:
(a) This is a simple perceptron which can only draw linear decision boundaries. **[3 marks]**
(b) This each hidden neuron learns a linear separation boundary and these are then combined by the output neuron to separate between inside and outside the two lines. **[3 marks]**
(c) The two neurons in the first hidden layer learn to draw a horizontal and a vertical line respectively, while the two neurons in the second hidden layer learn to combine these to create curves/boxes that separate e.g. top-left corner vs rest and bottom-right corner vs rest. Finally the output neuron combines these to separate between top-left and bottom-right vs rest. **[3 marks]**

B4. [18 marks] Consider CNN shown in Figure 3, which classifies 28 x 28 grayscale image inputs out of 10 possible classes.

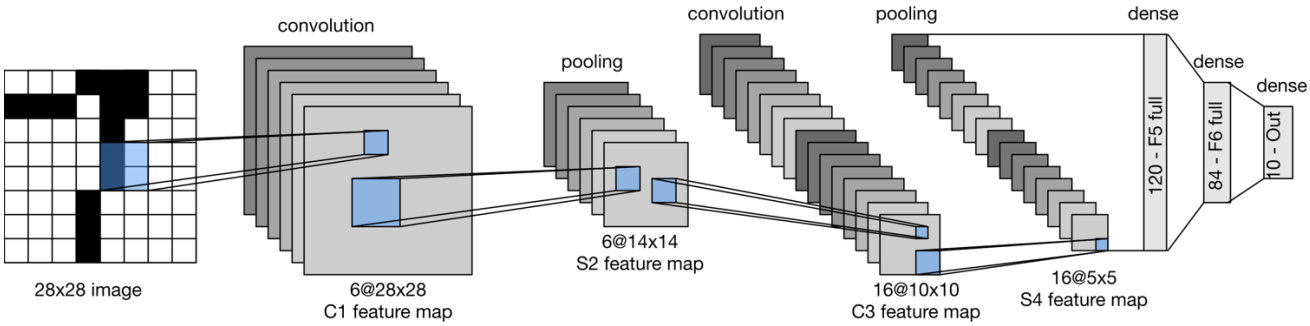


Figure 3: CNN architecture (LeNet).

Answer the following questions (showing all your calculations).

(a) Assuming no padding and stride = 1, what is the *number* of convolutional filters and their *size* for the first convolutional layer?
(b) What is the total number of learnable parameters for the first convolutional layer?
(c) How many biases are there in the first pooling layer?
(d) How many learnable parameters are there in the MLP (i.e., the fully connected network) that takes the S4 feature maps in input (i.e., the output of the last pooling layer) and outputs the probability of the 10 classes?

Answer box of B4

NOTE: Figure 3 shows a 28 x 28 input, however during the test I realized that this was a mistake and I told the student to change it to 32 x 32. Since some students may have no clearly understood my instructions, there are TWO valid answers to question B4(a) and B4(b), indicated below as a1 and a2, and b1 and b2 respectively.

(a1) The number of convolutional filters is 6 because the output of the first convolutional layer are 6 features maps. The input image has size 32 x 32 and the output feature maps have size 28 x 28, considering stride = 1 and no padding, the filter size must be 5 x 5. [**4 marks**]
(a2) The number of convolutional filters is 6 because the output of the first convolutional layer are 6 features maps. The input image has size 28 x 28 and the output feature maps have size 28 x 28, considering stride = 1 and no padding, the filter size must be 1 x 1. [**4 marks**]

(b1) The first convolutional layer has 6 x (5 x 5 + 1) = 156 learnable parameters. [**4 marks**]
(b2) The first convolutional layer has 6 x (1 x 1 + 1) = 12 learnable parameters. [**4 marks**]

(c) There are no biases (or learnable parameters) in the pooling layer. [**2 marks**]

(d) First MLP layer: 400 * 120 + 120 = 48,120. Second MLP layer: 120 * 84 + 84 = 10,080. Final MLP layer: 84 * 10 + 10 = 850. In total, 48,120+10,164+850=59,134 learnable parameters. [**8 marks**]